



Real-Time Pothole Detection

Using Data Mining & Data Warehousing

Project Report — DWDM Course

Course : Data Warehousing and Data Mining (DWDM)

Academic Year : 2025–2026

Technology : Python · YOLOv8 · SQLite · Streamlit · Scikit-learn

Date : October 05, 2026

Table of Contents

1.	Project Overview & Problem Statement	3
2.	Objectives	4
3.	System Architecture	4
4.	Technology Stack	5
5.	Dataset — RDD2022	6
6.	Data Preprocessing Pipeline	7
7.	YOLOv8 Object Detection	8
8.	Data Warehouse Design (Star Schema)	9
9.	OLAP Operations	11
10.	K-Means Data Mining	12
11.	Interactive Dashboard	13
12.	System Configuration	14
13.	Project Structure	14
14.	Demo vs Real Mode	15
15.	Limitations & Future Scope	16
16.	Conclusion	17

1. Project Overview & Problem Statement

This project is a college-level submission for the **Data Warehousing and Data Mining (DWDM)** course. It demonstrates a complete, end-to-end data engineering and machine learning pipeline applied to a real-world civil infrastructure problem: **pothole detection and analysis**.

1.1 Problem Statement

Potholes represent one of the most persistent and dangerous road infrastructure challenges in developing nations. Undetected or delayed-repair potholes lead to:

- **Road safety risks** — Thousands of accidents and injuries occur annually due to potholes.
- **Vehicle damage** — Suspension, tyre, and axle damage cost vehicle owners billions annually.
- **Economic loss** — Delayed pothole repairs escalate road maintenance costs significantly.
- **Urban planning gap** — Municipal authorities lack systematic, data-driven tools to prioritize repairs.

1.2 What This System Does (End-to-End)

#	Stage	Description
1	Data Acquisition	RDD2022 dataset — real road images with annotated potholes
2	Data Preprocessing	Clean, filter, convert, resize, augment, and split into train/val/test
3	Model Training	Train YOLOv8 object detection model on pothole class (D40)
4	Inference	Run YOLO on new images → bounding boxes, confidence scores, severity
5	Data Warehousing	Star Schema warehouse in SQLite via SQLAlchemy
6	OLAP Analysis	Roll-Up, Drill-Down, Slice, Dice on the data warehouse
7	Data Mining	K-Means clustering to discover pothole hotspot patterns
8	Dashboard	12-page interactive Streamlit dashboard with charts & maps

2. Objectives

- Detect potholes in road images using a state-of-the-art **YOLOv8 deep learning model**.
- Preprocess and clean the RDD2022 dataset using a structured pipeline (filtering, annotation conversion, resizing, augmentation).
- Design and implement a **Star Schema data warehouse** to store and manage detection records efficiently.
- Apply **OLAP operations** (Roll-Up, Drill-Down, Slice, Dice) to enable multi-dimensional analysis of pothole data.
- Apply **K-Means clustering** to mine pothole hotspots and identify patterns across locations and time periods.
- Estimate severity of each detected pothole using a project-defined heuristic based on bounding box size and confidence score.
- Build an **interactive dashboard** with Streamlit presenting all analytics, maps, and data mining results.
- Demonstrate real-world data engineering by integrating computer vision, data warehousing, OLAP, and data mining.

3. System Architecture

The system follows a layered data engineering architecture, transforming raw road images into actionable municipal intelligence through a series of tightly integrated processing stages.

Layer	Component	Role
Acquisition	RDD2022 Dataset	Source of real road images with XML annotations
Preprocessing	scripts/ pipeline	Clean → Filter D40 → Convert → Resize → Augment → Split
Modeling	YOLOv8 (Ultralytics)	Object detection: produces bounding boxes & confidence
Storage	SQLite + SQLAlchemy	Star Schema warehouse: FactDetection + 3 Dimension tables
Analysis	olap/ module	Roll-Up, Drill-Down, Slice, Dice operations
Mining	mining/ + scikit-learn	K-Means clustering for hotspot discovery
Presentation	Streamlit + Plotly + Folium	12-page interactive dashboard for end users

3.1 Data Flow Diagram

Each stage in the pipeline feeds the next, forming a clean ETL (Extract–Transform–Load) chain:

1. RDD2022 Dataset	Raw road images + Pascal VOC XML annotations
2. Preprocessing Pipeline	Clean → Filter D40 → XML→YOLO → Resize → Augment → Split 70/20/10
3. YOLOv8 Training & Inference	Transfer learning on pothole class; generates detection records
4. Data Warehouse (Star Schema)	FactDetection linked to DimLocation, DimTime, DimSeverity
5. OLAP Operations	Roll-Up / Drill-Down / Slice / Dice on the warehouse cube
6. K-Means Clustering	Feature engineering → StandardScaler → K-Means → hotspot labels
7. Streamlit Dashboard	12 interactive pages: maps, charts, OLAP tools, cluster visualisation

4. Technology Stack

Category	Technology	Version / Package
Language	Python	3.10+
Object Detection	Ultralytics YOLOv8	ultralytics ≥ 8.0.0
Deep Learning	PyTorch	torch ≥ 2.0.0
Image Processing	OpenCV	opencv-python ≥ 4.8.0
Image Processing	Pillow (PIL)	Pillow ≥ 10.0.0
Data Processing	Pandas	pandas ≥ 2.0.0
Numerical Computing	NumPy	numpy ≥ 1.24.0
Machine Learning	scikit-learn (K-Means)	scikit-learn ≥ 1.3.0
Database ORM	SQLAlchemy	SQLAlchemy ≥ 2.0.0
Database Engine	SQLite	Built-in with Python
Dashboard	Streamlit	streamlit ≥ 1.28.0
Charts & Visualisation	Plotly	plotly ≥ 5.17.0
Geospatial Maps	Folium + streamlit-folium	folium ≥ 0.14.0
Configuration	PyYAML	pyyaml ≥ 6.0
Logging	Loguru	loguru ≥ 0.7.0
Testing	pytest	pytest ≥ 7.4.0

5. Dataset — RDD2022

5.1 Overview

RDD2022 (Road Damage Dataset 2022) is a large-scale, publicly available benchmark dataset for road damage detection collected by researchers from Tohoku University, Japan. It contains road images from multiple countries including Japan, India, Czech Republic, Norway, and the United States.

- High-resolution road surface images captured from vehicles
- Bounding-box annotations in XML (Pascal VOC) format
- Multiple damage categories: longitudinal cracks, transverse cracks, alligator cracks, potholes
- Used in international **IEEE Big Data Challenge** competitions

5.2 Why RDD2022 Was Chosen

Criterion	Detail
Real-world data	Genuine road images, not synthetic
India subset available	Directly relevant to Indian road infrastructure context
Standardised annotations	Pascal VOC XML format, well-documented
Academically recognised	Used in IEEE Big Data Challenge competitions
Free and open	Publicly available for academic use
Pothole class included	Contains the D40 class specifically for potholes

5.3 Damage Classification — Only D40 Used

Code	Damage Type	Description	Used
D00	Longitudinal Crack	Cracks along the road direction	■ Filtered out
D10	Transverse Crack	Cracks across the road direction	■ Filtered out
D20	Alligator Crack	Interconnected cracking pattern	■ Filtered out
D40	Pothole	Cavity/hole in road surface	■ Primary class

This project focuses exclusively on the D40 (pothole) class. All other classes are filtered out during preprocessing.

5.4 Expected Dataset Structure

```
dataset/  
  rdd2022/  
    India/
```

```
■■■ images/  
■ ■■■ train/  
■ ■■■ India_000001.jpg  
■ ■■■ ...  
■■■ annotations/  
■■■ xmls/  
■■■ India_000001.xml  
■■■ ...
```


6. Data Preprocessing Pipeline

The preprocessing pipeline transforms raw RDD2022 data into a clean, YOLO-ready dataset. Each step is implemented as a separate, reusable Python script under the `scripts/` directory.

Step 1 — CLEANING

`scripts/clean_dataset.py`

- Remove corrupt/unreadable images
- Remove XML files with no valid annotations
- Remove duplicate images using perceptual hashing (imagehash)
- Log all removed files

Step 2 — FILTERING

`scripts/filter_d40.py`

- Parse each XML annotation file
- Retain only images that contain at least one D40 (pothole) label
- Discard images with only D00/D10/D20 labels
- Produces a focused pothole-only subset

Step 3 — ANNOTATION CONVERSION

`scripts/convert_annotations.py`

- Convert Pascal VOC XML format → YOLO TXT format
- YOLO format: (normalised 0–1)
- Class ID for D40 (pothole) = 0
- One .txt file created per image

Step 4 — RESIZING

`scripts/resize_images.py`

- Resize all images to 416×416 pixels (YOLOv8 default input size)
- Maintain aspect ratio with letterboxing to avoid distortion
- Update bounding box coordinates accordingly

Step 5 — AUGMENTATION

`utils/preprocessing.py`

- Horizontal flip (mirror) — increases variety
- Random brightness/contrast adjustments — handles lighting variation
- Slight rotation ($\pm 5^\circ$) — handles camera angle variation
- Applied only to training split to prevent data leakage

Step 6 — SPLITTING

`utils/preprocessing.py`

- Randomly shuffle the filtered dataset
- Split: 70% Training / 20% Validation / 10% Test
- Output structure placed in `data/` for YOLO training config

6.1 The 70 / 20 / 10 Train-Validation-Test Split

Split	Proportion	Purpose
Training Set	70%	Used to train the YOLOv8 model weights
Validation Set	20%	Used during training to tune hyperparameters and monitor overfitting
Test Set	10%	Held-out set used only at the end to report final model performance

7. YOLOv8 Object Detection

7.1 What is YOLOv8?

YOLO (You Only Look Once) is a family of real-time object detection models. **YOLOv8** by Ultralytics (2023) is the latest generation, offering single-pass detection — the entire image is processed in one forward pass through the neural network, making it extremely fast. Its anchor-free architecture improves detection of small and irregularly shaped objects like potholes.

7.2 Training Configuration

Hyperparameter	Value	Description
epochs	50	Number of full passes over the training data
batch_size	16	Images processed per gradient update
imgsz	416	Input image size (pixels) for the model
conf	0.20	Minimum detection confidence threshold
iou_threshold	0.45	IoU threshold for Non-Maximum Suppression (NMS)
device	cpu	Inference device (cpu / cuda / mps)
base_model	yolov8n.pt	Pre-trained nano model (fine-tuned)

7.3 Severity Heuristic

Each detection is assigned a severity label based on a project-defined heuristic combining **bounding box area ratio** and **YOLO confidence score**:

Severity	Score	Criteria
LOW	1	Small bounding box (< 1% image area) AND low confidence (< 0.40)
MEDIUM	2	Moderate area or moderate confidence (between thresholds)
HIGH	3	Large bounding box (≥ 5% image area) OR high confidence (≥ 0.70)
CRITICAL	4	Large bounding box AND high confidence simultaneously

Note: Severity is a project-defined heuristic. It is NOT an official label from the RDD2022 dataset or any validated severity standard.

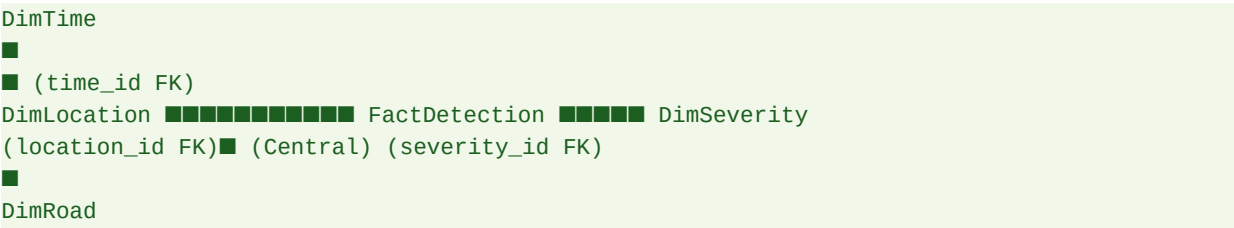
7.4 Model Files

File / Script	Purpose
ml/train_yolo.py	YOLOv8 training script — loads base model, fine-tunes on pothole dataset
ml/train_rdd2022.py	Alternative training script tailored to RDD2022 dataset specifics
ml/evaluate_yolo.py	Model evaluation — computes mAP, precision, recall on test set
ml/predict_image.py	Inference on single images — produces annotated output images
ml/predict_video.py	Inference on video files — frame-by-frame pothole detection
ml/live_detection.py	Real-time webcam/camera inference with live bounding box display
models/pothole_yolo.pt	Saved trained weights (generated after training)

8. Data Warehouse Design (Star Schema)

The data warehouse uses a **Star Schema** — the industry-standard structure for OLAP and analytical workloads. It consists of one central fact table surrounded by dimension tables, enabling fast multi-dimensional queries.

8.1 Schema Diagram



8.2 FactDetection Table (Central Fact Table)

One row per detected pothole instance.

Column	Type	Description
detection_id	INTEGER PK	Unique detection identifier (auto-increment)
location_id	INTEGER FK	Foreign key → DimLocation
time_id	INTEGER FK	Foreign key → DimTime
severity_id	INTEGER FK	Foreign key → DimSeverity
confidence_score	FLOAT	YOLO model confidence score (0.0 – 1.0)
bbox_area_px	FLOAT	Bounding box area in pixels
image_filename	TEXT	Source image filename for traceability

8.3 DimLocation (Spatial Dimension)

Column	Type	Description
location_id	INTEGER PK	Unique location surrogate key
latitude	FLOAT	GPS latitude (simulated / future real GPS)
longitude	FLOAT	GPS longitude (simulated / future real GPS)
city	TEXT	City name (e.g., Mumbai, Bangalore)

Column	Type	Description
district	TEXT	District name
road_name	TEXT	Road identifier
zone	TEXT	Zone/area label (North/South/East/West/Central)

8.4 DimTime (Temporal Dimension — OLAP Hierarchy)

Column	Type	Description
time_id	INTEGER PK	Unique time surrogate key
timestamp	DATETIME	Full detection timestamp
hour	INTEGER	Hour of day (0–23) — for drill-down to hour level
day	INTEGER	Day of month
month	INTEGER	Month (1–12)
quarter	INTEGER	Quarter (1–4) — for roll-up to quarter level
year	INTEGER	Year — for roll-up to annual level
day_of_week	TEXT	e.g., Monday, Tuesday
is_weekend	BOOLEAN	True if Saturday or Sunday

8.5 DimSeverity (Severity Dimension)

Column	Type	Description
severity_id	INTEGER PK	Unique severity surrogate key
severity_label	TEXT	LOW / MEDIUM / HIGH / CRITICAL
severity_score	INTEGER	Numeric score (1–4) for ordering and aggregation
description	TEXT	Human-readable description of the severity level

9. OLAP Operations

OLAP (Online Analytical Processing) operations enable multi-dimensional analysis of the data warehouse without writing complex SQL queries. The project implements all four core OLAP operations.

Roll-Up (olap/rollup.py)

Aggregates data from a lower level to a higher level of a dimension hierarchy.

Example: Detection counts aggregated: Day → Month → Quarter → Year

Use case: How many potholes were detected per month this year?

Drill-Down (olap/drilldown.py)

The reverse of Roll-Up — navigates from a higher level to a lower level for more detail.

Example: Zoom from City → District → Road → Specific Location

Use case: Which specific road in Bangalore has the most potholes this quarter?

Slice (olap/slice.py)

Selects a single value for one dimension and analyses the remaining dimensions.

Example: Fix severity = 'CRITICAL'; analyse detections across all locations and times

Use case: Show me all critical potholes regardless of location or time.

Dice (olap/dice.py)

Selects a sub-cube by applying filters on two or more dimensions simultaneously.

Example: Filter to city = 'Mumbai' AND month = 'June' AND severity = 'HIGH'

Use case: Show me high-severity potholes in Mumbai during June.

10. K-Means Data Mining

K-Means is an unsupervised machine learning algorithm applied here to discover **pothole hotspots** — geographic and temporal clusters with high detection density, enabling prioritised repair scheduling.

10.1 Feature Engineering

Feature	Source	Description
latitude	DimLocation	Spatial coordinate — north-south position
longitude	DimLocation	Spatial coordinate — east-west position
hour_of_day	DimTime	Time of detection (0–23)
day_of_week	DimTime	Day as integer (0 = Monday, 6 = Sunday)
severity_score	DimSeverity	Numeric severity (1–4)
confidence_score	FactDetection	YOLO confidence (0.0–1.0)

10.2 Preprocessing & Algorithm

- **Normalisation:** All features normalised using StandardScaler (zero mean, unit variance) before clustering.
- **Algorithm:** K-Means (sklearn.cluster.KMeans) with k-means++ initialisation.
- **K Selection:** Optimal K determined using the **Elbow Method** (plotting inertia vs. K values).
- **Multiple Runs:** n_init=10 random initialisations to avoid local minima.
- **Default K:** 3 clusters (configurable in config.yaml).

10.3 Cluster Interpretation

Cluster Profile	Characteristics	Interpretation
Priority 1 Hotspot	High density, high severity, central zone	Immediate repair needed — highest priority
Maintenance Zone	Medium density, mixed severity, mid-zone	Regular maintenance scheduled area
Low-Priority Zone	Low density, low severity, peripheral zone	Monitor only — no immediate action needed

10.4 Mining Module Files

File	Purpose
mining/prepare_features.py	Feature engineering — queries warehouse, builds feature matrix

File	Purpose
mining/kmeans.py	K-Means clustering, elbow method, cluster profiling
mining/cluster_analysis.py	Cluster interpretation and visualisation helpers
mining/ai_analytics.py	Advanced analytical functions over cluster results

11. Interactive Dashboard

The interactive dashboard is built with **Streamlit** and **Plotly**. It provides **12 pages** accessible from the sidebar, covering all aspects of the system from real-time detection summaries to OLAP tools and cluster visualisations.

#	Page	Description
1	■ Home / Overview	Project summary, key metrics (total detections, hotspots, severity breakdown), system status
2	■ Detection Summary	Total detection counts, daily/weekly trends, confidence score distribution histogram
3	■■ Pothole Map	Interactive Folium map showing all potholes colour-coded by severity
4	■ Time Analysis	OLAP Roll-Up charts — detections by hour, day, month, quarter, year
5	■ Location Analysis	OLAP Drill-Down from city → district → road; top roads by detection count
6	■■ Severity Analysis	OLAP Slice — filter by severity level; pie chart, bar chart of severity distribution
7	■ OLAP Dice	Interactive OLAP Dice tool — user selects dimension filters to query sub-cubes
8	■ Hotspot Clusters	K-Means results — scatter map of clusters, elbow curve, cluster profiles
9	■ Trend Analysis	Long-term detection trends; peak hours and peak seasons identification
10	■■ Detection Gallery	Browse sample detection images with bounding boxes and confidence annotations
11	■■ Data Pipeline Status	Step-by-step pipeline status: dataset ready? model trained? warehouse populated?
12	■■ About / Docs	Project description, tech stack, academic information, demo data disclaimer

Access the dashboard by running:

```
streamlit run app.py
```

The dashboard opens automatically at **http://localhost:8501**.

12. System Configuration

All system parameters are centralised in **config.yaml** at the project root. Key configurable sections:

Section	Key Parameters
dataset	rdd2022_path, processed_path, split ratios (70/20/10)
model	yolo_model_path, image_size (416), confidence_threshold (0.20), iou_threshold (0.45)
severity	area_high/low thresholds, conf_high/low thresholds, score mapping
augmentation	horizontal_flip, brightness_range [0.7–1.3], scale_range [0.8–1.2]
database	type (sqlite), sqlite_path, MySQL settings (optional)
kmeans	n_clusters (3), random_state (42), max_iter (300)
camera	device index (0), frame_skip (5), save_detections_only (true)
training	epochs (50), batch_size (16), workers (2)

13. Project Structure

Directory / File	Description
app.py	Main Streamlit dashboard entry point
config.yaml	Project-wide configuration
requirements.txt	Python dependencies
scripts/clean_dataset.py	Step 1: Remove corrupt/duplicate images
scripts/filter_d40.py	Step 2: Keep only D40 (pothole) annotations
scripts/convert_annotations.py	Step 3: XML → YOLO TXT format conversion
scripts/resize_images.py	Step 4: Resize images to 416×416
scripts/prepare_dataset.py	Master script: runs all preprocessing steps
ml/train_yolo.py	YOLOv8 training script
ml/evaluate_yolo.py	Model evaluation (mAP, precision, recall)
ml/live_detection.py	Real-time webcam pothole detection
warehouse/models.py	SQLAlchemy ORM models (Star Schema tables)
warehouse/database.py	DB connection, initialisation, session management
warehouse/ingestion.py	Ingest detection records into the warehouse
warehouse/queries.py	Reusable warehouse query functions
warehouse/seed_demo_data.py	Seed synthetic demo data for demo mode
olap/rollup.py	Roll-Up aggregation functions
olap/drilldown.py	Drill-Down navigation functions
olap/slice.py	Slice (single-dimension filter) functions
olap/dice.py	Dice (multi-dimension filter) functions
mining/prepare_features.py	Feature engineering for K-Means clustering
mining/kmeans.py	K-Means clustering, elbow method, profiling
utils/preprocessing.py	Augmentation and train/val/test split logic
utils/severity.py	Severity classification heuristic
tests/	Unit and integration test suite

Directory / File	Description
dataset/rdd2022/	RDD2022 dataset (not committed to git — large)
models/pothole_yolo.pt	Trained YOLOv8 weights (generated after training)
data/warehouse.db	SQLite data warehouse file
logs/	Training and pipeline log files

14. Demo vs Real Mode

Aspect	Demo Mode	Real Mode
Data Source	Synthetic (~500 records from seed_demo_data.py)	Real inference from RDD2022 + trained YOLO
Model Required	No model needed	Requires models/pothole_yolo.pt
Dataset Required	Not required	RDD2022 India subset at dataset/rdd2022/
Dashboard	Fully functional (all 12 pages)	Fully functional with real detections
Use Case	Quick demo, course evaluation	Research, production evaluation
Caution	Data is NOT real — for demo only	Data is real RDD2022 inference results

14.1 Running Demo Mode

```
python warehouse/seed_demo_data.py
streamlit run app.py
```

14.2 Running Real Mode

- 1. Download RDD2022 → place at dataset/rdd2022/
- 2. python scripts/prepare_dataset.py
- 3. python ml/train_yolo.py
- 4. streamlit run app.py

15. Limitations & Future Scope

15.1 Current Limitations

Limitation	Details
No trained model included	models/pothole_yolo.pt not committed (file size). User must train.
GPS unavailable	Real GPS coordinates require hardware. Location data is simulated.
Severity is project-defined	LOW/MEDIUM/HIGH/CRITICAL classification is a heuristic, not an official standard.
Demo data is synthetic	Demo mode seed data is randomly generated — not real-world detections.
Single damage class	Only D40 (pothole) class used. Cracks etc. are filtered out.
SQLite database	SQLite used for portability. Production deployments need PostgreSQL/MySQL.
No real-time video stream	Current system processes individual images. Video streaming needs extra work.

15.2 Future Scope

Enhancement	Description
GPS Integration	Integrate vehicle-mounted GPS to record exact pothole coordinates automatically
Mobile Application	Android/iOS app allowing citizens to report potholes with photos and auto-GPS
Cloud Deployment	Deploy on AWS/GCP/Azure with cloud-hosted database for city-wide use
Real-Time Alert System	Automatic SMS/email alerts to municipal authorities when severity clusters spike
More Damage Classes	Extend YOLO to detect D00/D10/D20 (cracks) in addition to D40 (potholes)
Repair Tracking	Add workflow: Reported → Assigned → Under Repair → Fixed
Predictive Analytics	ARIMA/LSTM time-series forecasting for future pothole formation hotspots
Road Authority API	REST API layer for programmatic access by municipal dashboards

16. Conclusion

This project successfully demonstrates a **complete, end-to-end data engineering and machine learning pipeline** applied to the real-world problem of pothole detection and analysis. By integrating computer vision (YOLOv8), relational data warehousing (Star Schema / SQLite), OLAP operations, unsupervised data mining (K-Means), and interactive visualisation (Streamlit), the system covers the full spectrum of DWDM course concepts.

The project achieves all stated objectives: automated pothole detection from road images, structured storage in a dimensional data warehouse, multi-dimensional OLAP analysis, hotspot discovery through clustering, and presentation through a polished interactive dashboard. The modular architecture ensures each component is independently testable and extendable.

Key Concepts Demonstrated

DWDM Topic	Implementation
ETL Pipeline	Extract (RDD2022) → Transform (preprocessing) → Load (warehouse ingestion)
Star Schema	FactDetection + DimLocation + DimTime + DimSeverity
OLAP Cube Operations	Roll-Up, Drill-Down, Slice, Dice implemented in olap/ module
Unsupervised Learning	K-Means clustering for pothole hotspot discovery
Transfer Learning	Fine-tuning pre-trained YOLOv8n on domain-specific RDD2022 dataset
Data Visualisation	12-page Streamlit dashboard with Plotly charts and Folium maps

Academic Information

Field	Details
Course	Data Warehousing and Data Mining (DWDM)
Academic Year	2025–2026
Institution	[Your Institution Name]
Department	[Your Department]
Guide	[Faculty / Supervisor Name]
Team Members	[Student Name(s) and Roll Numbers]
Dataset	RDD2022 (Road Damage Dataset 2022) — Tohoku University, Japan

Field	Details
Report Date	October 05, 2026

Disclaimer: Demo mode data is synthetically generated and does NOT represent real RDD2022 inference results. The RDD2022 dataset is subject to its own license — refer to the official repository for terms of use.